سند جامع بکند کاروانسرا (Caravansara)

Nov 1, 2026 · @zia

خلاصه

این سند بکند کاروانسرا (Caravansara)، یک سایت رزرو اقامتگاه شبیه Airbnb، را تعریف می‌کند: مهمان جستجو و رزرو می‌کند، میزبان آگهی می‌گذارد، و دو نفر هرگز نمی‌توانند یک اقامتگاه را برای شب‌های مشترک بگیرند.

بکند با Python و FastAPI، به‌صورت یک برنامه‌ی ماژولار (نه میکروسرویس) روی PostgreSQL ساخته شده است. کد در پوشه‌ی `backend/` است و نحوه‌ی اجرای آن در `backend/README.md` آمده است.

- **مخاطب:** توسعه‌دهندگان و هر کسی که دامنه‌ی نسخه‌ی اول را تأیید می‌کند.
- **دامنه:** فقط بکند و API. رابط کاربری جزو این کار نیست.
- **منابع:** `search_query.sql`، `schema.sql`، `CONTEXT.md`، دموی ERD و تصمیم‌های گرفته‌شده در بررسی مشترک. نسخه‌ی قدیمی Design Overview مبنا نیست، چون هنوز پرداخت، پیام و پاسخ میزبان را شرح می‌دهد.

تصمیم‌های کلیدی و دلیل آن‌ها

مهم‌ترین تصمیم این است که قوانین حیاتی در خود دیتابیس باشند، چون چند نمونه‌ی API یا کد اشتباه نمی‌توانند آن‌ها را دور بزنند. ستون «منشأ» نشان می‌دهد هر تصمیم از کجا آمده است.

تصمیم	دلیل	منشأ
معماری ماژولار، نه میکروسرویس	رزرو و نظر در یک تراکنش می‌مانند و استقرار ساده است؛ مرز ماژول‌ها با import-linter حفظ می‌شود	درخواست مالک پروژه
جلوگیری از رزرو تکراری با `EXCLUDE`	بررسی و ثبت در برنامه دو قدم جداست و مسابقه می‌سازد؛ دیتابیس تنها نقطه‌ی مشترک همه‌ی سرورهاست	طراحی قبلی
جستجوی شعاعی با PostGIS	فاصله روی کره محاسبه دقیق می‌خواهد؛ `ST_DWithin` با ایندکس GiST این را آماده می‌دهد و مشکل خط ۱۸۰ درجه را هم ندارد. این تصمیم جای «بدون PostGIS» قبلی را می‌گیرد	درخواست مالک پروژه، انتخاب ابزار از Claude
ذخیره‌ی نقطه‌ی عمومی جدا از مختصات واقعی	اگر فیلتر روی مختصات واقعی باشد، با کوچک کردن شعاع مکان دقیق پیدا می‌شود و جابه‌جایی پین بی‌اثر است	پیشنهاد Claude
سقف ۱ تا ۱۰۰ کیلومتر برای شعاع	شعاع خیلی بزرگ کوئری را کند می‌کند و شعاع خیلی کوچک فقط درخواست‌های بی‌فایده می‌سازد	پیشنهاد Claude
پرداخت شبیه‌سازی‌شده	چرخه‌ی نگه‌داشت و انقضا کامل تست می‌شود و بعداً فقط درگاه واقعی جایگزین می‌شود	مالک پروژه، به پیشنهاد Claude

تصمیم	دلیل	منشأ
JWT با refresh token	برای وب و موبایل مناسب است و برخلاف JWT تنها، خروج واقعی و ابطال دارد	مالک پروژه، به پیشنهاد Claude
MinIO با S3 API	همان کد در توسعه، تست و تولید اجرا می‌شود و تست آپلود واقعی است	مالک پروژه، به پیشنهاد Claude
چهار اصلاح اسکیما	هر کدام مشکل مشخصی را حل می‌کند که در بخش پایگاه داده آمده است	مالک پروژه، به پیشنهاد Claude
انقضای نگه‌داشت در همان تراکنش رزرو	اگر فقط worker این کار را بکند، تا دقیقه‌ی بعد تاریخ بی‌دلیل بسته می‌ماند	پیشنهاد Claude
پرداخت با `UPDATE` شرطی	اگر پرداخت و انقضا هم‌زمان برسند، فقط یکی برنده می‌شود	پیشنهاد Claude
تکرار رزرو همان رزرو را برمی‌گرداند	کاربر با قطع شبکه باید نتیجه‌ی موفق را ببیند، نه خطا	پیشنهاد Claude
ماشین حالت صریح در service	دیتابیس گذار وضعیت را کنترل نمی‌کند، پس `completed` به `pending` جلوی دیتابیس آزاد است	پیشنهاد Claude
worker جدا از API	نمونه‌های API بدون حالت می‌مانند و زمان‌بندی با تعداد آن‌ها چند برابر نمی‌شود	پیشنهاد Claude
بدون Redis	کاری نداریم که آن را بهتر از PostgreSQL انجام دهد؛ جزئیات در بخش معماری آمده است	پیشنهاد Claude
Rate limit برای ورود و ثبت‌نام	بدون آن حدس زدن رمز عبور محدودیتی ندارد	پیشنهاد Claude
SQLAlchemy async	کار این API بیشتر انتظار برای دیتابیس است و FastAPI با async هم‌خوان است	پیشنهاد Claude
تست روی PostgreSQL واقعی	`EXCLUDE`، تریگرها و PostGIS با mock یا SQLite قابل تست نیستند	پیشنهاد Claude
میزبان آگهی خودش را رزرو نمی‌کند	جلوی ساخت امتیاز جعلی برای خود را می‌گیرد	مالک پروژه
نظر غیرقابل ویرایش و مهلت ۱۴ روزه	اعتبار امتیازها حفظ می‌شود و نظرها به اقامت نزدیک می‌مانند	مالک پروژه
لغو بدون جریمه تا قبل از روز ورود	پرداخت واقعی نداریم، پس جریمه و بازپرداخت معنی ندارد	مالک پروژه
کاتالوگ فقط با seed	نقش ادمین نداریم و فهرست کنترل‌شده باید کنترل‌شده بماند	مالک پروژه

قابلیت‌های سایت

نسخه‌ی اول چرخه‌ی کامل مهمان و میزبان را پوشش می‌دهد: از ثبت‌نام تا جستجو، رزرو، اقامت و نظر. مانده‌ی کار در بخش «خارج از دامنه» آمده است.

حساب کاربری

قابلیت	توضیح
ثبت‌نام و ورود	با ایمیل و رمز عبور؛ ایمیل بدون حساسیت به حروف یکتاست
نشست امن	JWT کوتاه‌عمر و refresh token قابل ابطال؛ خروج واقعی از حساب
پروفایل	نام و تصویر؛ هر کاربر هم مهمان است و هم می‌تواند میزبان باشد
محدودیت تلاش	rate limit برای ورود و ثبت‌نام

مهمان

قابلیت	توضیح
جستجوی «کجا»	انتخاب کشور و شهر با پیشنهاد خودکار، یا جستجوی شعاعی با مرکز و شعاع
جستجوی «کی»	تاریخ ورود و خروج؛ فقط اقامتگاه‌های آزاد نمایش داده می‌شوند
جستجوی «چند نفر»	بزرگسال، کودک، نوزاد و حیوان خانگی؛ نوزاد در ظرفیت حساب نمی‌شود
فیلتر امکانات	آگهی باید همه‌ی امکانات انتخاب‌شده را داشته باشد، با شمارش برای پنل فیلتر
نتایج و نقشه	کارت با عکس کاور، امتیاز، قیمت کل و پین تقریبی روی نقشه
صفحه‌ی آگهی	عکس‌ها، توضیحات، امکانات دسته‌بندی‌شده، ظرفیت، قیمت و نظرها
رزرو	نگه‌داشت ۱۵ دقیقه‌ای تاریخ‌ها و سپس پرداخت شبیه‌سازی‌شده
مدیریت سفرها	فهرست رزروها، جزئیات و آدرس دقیق پس از تأیید، و لغو تا قبل از روز ورود
ثبت نظر	امتیاز ۱ تا ۵، شش امتیاز جزئی و متن، تا ۱۴ روز بعد از خروج

میزبان

قابلیت	توضیح
مدیریت آگهی	ساخت، ویرایش، فعال و غیرفعال کردن آگهی
مدیریت عکس‌ها	آپلود مستقیم با presigned URL، تعیین کاور، تغییر ترتیب، عنوان و برچسب اتاق
انتخاب امکانات	از کاتالوگ ۱۲ دسته و ۳۱ امکان

قابلیت	توضیح
مشاهده‌ی رزروها	فهرست رزروهای هر آگهی
لغو رزرو	تا قبل از روز ورود

سیستم

قابلیت	توضیح
جلوگیری از رزرو هم‌زمان	محدودیت `EXCLUDE` در PostgreSQL
ایمنی در تکرار رزرو	کلاینت در هدر `Idempotency-Key` می‌فرستد؛ تکرار درخواست رزرو دوم نمی‌سازد
انقضای نگه‌داشت	نگه‌داشت پرداخت‌نشده `expired` می‌شود و تاریخ‌ها آزاد می‌شوند
پایان اقامت	رزرو بعد از ساعت خروج به وقت شهر `completed` می‌شود
به‌روز ماندن امتیاز	تریگر `avg_rating` و `review_count` را همگام می‌کند
داده‌ی اولیه	فرمان seed برای کشور، شهر و امکانات (فقط ردیف‌های تازه)، فرمان جدا برای داده‌ی دمو با عکس، و `seed-bulk` برای داده‌ی حجیم در دیتابیس جدا

قواعد کسب‌وکار

هر قانون هر جا که ممکن باشد در دیتابیس اعمال می‌شود و بقیه در لایه‌ی service. ستون آخر محل اجرای هر قانون را نشان می‌دهد.

حوزه	قانون	محل اجرا
رزرو	دو رزرو فعال یک آگهی هم‌پوشانی ندارند	دیتابیس (`EXCLUDE`)
رزرو	ورود همان روز خروج قبلی مجاز است	دیتابیس (بازه‌ی نیمه‌باز)
رزرو	میزبان نمی‌تواند آگهی خودش را رزرو کند	service
رزرو	بزرگسال و کودک حداکثر `max_guests`؛ نوزاد حساب نمی‌شود	service
رزرو	حیوان خانگی فقط با `pets_allowed` و تا `max_pets`	service
رزرو	ورود از امروز به وقت شهر به بعد	service
رزرو	اقامت حداکثر ۳۶۵ شب و حداکثر ۱ سال جلوتر؛ هر دو قابل تنظیم	service

حوزه	قانون	محل اجرا
قیمت	جمع = شب‌ها × `price_per_night` + `cleaning_fee`؛ در لحظه‌ی رزرو روی رزرو کپی می‌شود	service
پرداخت	فقط مهمان همان رزرو و فقط تا پایان مهلت نگه‌داشت	service (`UPDATE` شرطی)
لغو	مهمان و میزبان تا قبل از روز `check_in`؛ بدون جریمه و بازپرداخت	service
لغو	رزرو لغوشده تاریخ‌ها را آزاد می‌کند	دیتابیس
نظر	فقط برای رزرو `completed` و حداکثر یک نظر برای هر رزرو	دیتابیس (تریگر و `UNIQUE`)
نظر	فقط مهمان همان رزرو درباره‌ی همان آگهی	دیتابیس (کلید خارجی ترکیبی)
نظر	حداکثر ۱۴ روز بعد از `check_out`	service
نظر	بعد از ثبت ویرایش یا حذف نمی‌شود	نبود endpoint
آگهی	حذف نمی‌شود و فقط با `is_active` غیرفعال می‌شود؛ رزروهای قبلی برقرار می‌مانند	service
حریم خصوصی	آدرس دقیق فقط برای میزبان و مهمان رزرو `confirmed` یا `completed`	service
حریم خصوصی	جستجو، فاصله و پین فقط از نقطه‌ی عمومی جابه‌جاشده استفاده می‌کنند تا مکان واقعی با میانگین‌گیری یا کوچک کردن شعاع لو نرود	service
جستجو	شهر یا مرکز و شعاع الزامی است؛ شعاع بین ۱ تا ۱۰۰ کیلومتر و قابل تنظیم است؛ تاریخ ورود و خروج با هم می‌آیند	service
جستجو	مرتب‌سازی پیش‌فرض با امتیاز و سپس تعداد نظر، و در جستجوی شعاعی با فاصله هم ممکن است؛ صفحه‌بندی با limit/offset	service
جستجو	نگه‌داشت `pending` فقط تا `hold_expires_at` تاریخ‌ها را می‌بندد، حتی پیش از اجرای worker؛ وگرنه اقامتگاه آزاد تا یک دقیقه گرفته نشان داده می‌شد	service (کوئری جستجو)

جستجوی شعاعی

کاربر مرکز و شعاع می‌فرستد و آگهی‌های داخل آن دایره برمی‌گردند. این روش جای محدوده‌ی مستطیلی قبلی را می‌گیرد؛ جستجو با شهر هم باقی می‌ماند.

ذخیره‌سازی

داده	ستون	چه کسی می‌بیند	دلیل
مختصات واقعی	`latitude`، `longitude`	میزبان و مهمان رزرو قطعی	از طراحی قبلی می‌ماند و در هیچ فیلتر عمومی استفاده نمی‌شود
نقطه‌ی عمومی	`public_location geography(Point, 4326)`	همه	`geography` فاصله را به متر و روی کره حساب می‌کند

نقطه‌ی عمومی هنگام ساخت آگهی با جابه‌جایی تصادفی حدود ۱۰۰ تا ۵۰۰ متر ساخته و ذخیره می‌شود. فقط وقتی مختصات واقعی تغییر کند دوباره محاسبه می‌شود. دلیل ذخیره‌کردن این است که محاسبه‌ی تازه برای هر درخواست با میانگین‌گیری لو می‌رود و با ایندکس هم قابل جستجو نیست.

کوئری

```
CREATE INDEX listings_public_location_idx
    ON listings USING gist (public_location) WHERE is_active;

SELECT l.id,
       ST_Distance(l.public_location, c.center) / 1000 AS distance_km
FROM   listings l,
       (SELECT ST_MakePoint(:lng, :lat)::geography AS center) c
WHERE  l.is_active
  AND  ST_DWithin(l.public_location, c.center, :radius_km * 1000);
```

بقیه‌ی فیلترها مثل تاریخ، ظرفیت و امکانات طبق `search_query.sql` به این کوئری اضافه می‌شوند.

قرارداد API

`GET /search/listings?lat=60.17&lng=24.94&radius_km=5` از این روش استفاده می‌کند.

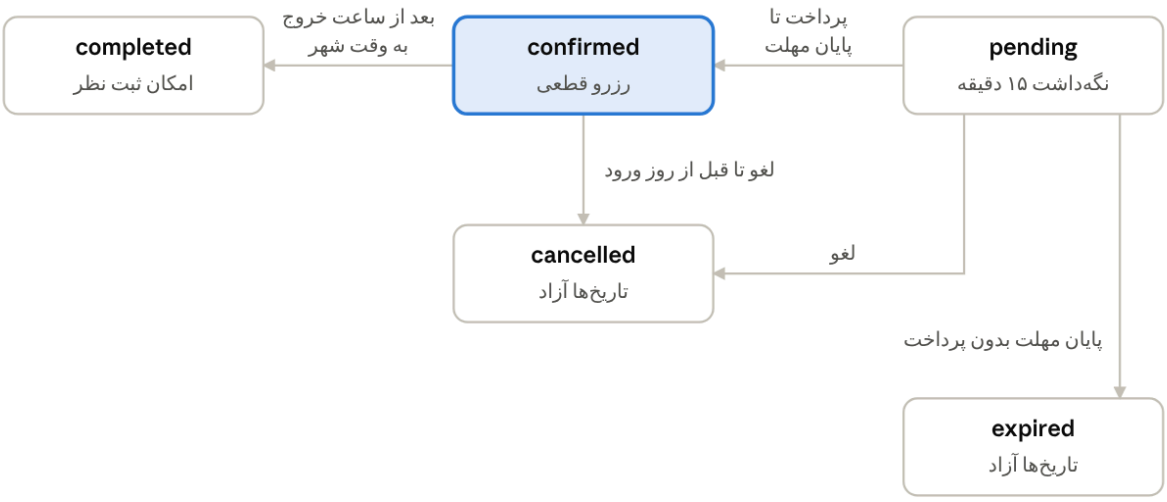
- **ورودی:** `city_id` یا هر سه مقدار `lng` ،`lat` و `radius_km`. از این دو روش باید یکی فرستاده شود.
- **شعاع:** بازه‌ی ۱ تا ۱۰۰ کیلومتر و قابل تنظیم.
- **خروجی:** نقطه‌ی عمومی و `distance_km` گردشده به ۰٫۱ کیلومتر؛ فاصله هم از نقطه‌ی عمومی حساب می‌شود.
- **مرتب‌سازی:** پیش‌فرض امتیاز است و `sort=distance` هم مجاز است.

اثر جانبی این تغییر: `ST_MakePoint` ابتدا طول و بعد عرض می‌گیرد، پس ترتیب پارامترها تست می‌شود و تصویر Docker دیتابیس هم باید PostGIS داشته باشد.

چرخه‌ی رزرو و جریان‌های اصلی

رزرو با نگه‌داشت ۱۵ دقیقه‌ای شروع می‌شود و فقط با پرداخت در مهلت قطعی می‌شود. گذارهای دیگر رد می‌شوند و خطای `invalid_booking_state` برمی‌گردد.

فقط پرداخت در مهلت، نگه‌داشت را به رزرو قطعی تبدیل می‌کند



`expired` و `cancelled` تاریخ‌ها را آزاد می‌کنند، چون محدودیت `EXCLUDE` فقط `confirmed`، `pending` و `completed` را می‌شمارد.

ساخت رزرو

1. کلاینت `POST /bookings` را با یک `Idempotency-Key` تازه می‌فرستد.
2. اگر همان کلید از همان کاربر با همان محتوا قبلاً ثبت شده، همان رزرو برمی‌گردد.
3. service ظرفیت، تاریخ‌ها، مالکیت آگهی و سقف اقامت را کنترل می‌کند.
4. در همان تراکنش، نگه‌داشت‌های منقضی‌شده‌ی همان آگهی `expired` می‌شوند تا تاریخ بی‌دلیل بسته نماند.
5. قیمت محاسبه و ردیف `pending` با `hold_expires_at` درج می‌شود.
6. برنده‌ی رقابت 201 می‌گیرد و بازنده با خطای `23P01` پاسخ 409 می‌گیرد.

کارهای زمان‌بندی‌شده

worker هر دقیقه دو کار انجام می‌کند: نگه‌داشت‌های گذشته از مهلت را `expired` می‌کند و رزروهایی را که ساعت خروجشان به وقت شهر گذشته `completed` می‌کند. اجرای هم‌زمان دو نمونه‌ی worker به داده آسیب نمی‌زند، چون هر تغییر یک `UPDATE` شرطی است.

آپلود عکس

1. میزبان از API یک presigned URL می‌گیرد.
2. مرورگر فایل را مستقیم در فضای ذخیره‌سازی می‌گذارد.
3. کلاینت `storage_key` و ابعاد را به API می‌فرستد و بکند پس از بررسی وجود فایل، ردیف را ثبت می‌کند.
4. نشانی CDN هنگام پاسخ از روی `storage_key` ساخته می‌شود.

معماری و پشته‌ی فنی

برنامه یک کدبیس و یک دیتابیس دارد و به ماژول‌های با مرز مشخص تقسیم می‌شود؛ از API و worker دو پروسه اجرا می‌شود. رزرو قلب سیستم است و در نمودار مشخص شده.



هر ماژول `models`، `repository`، `service`، `router` و `schemas` خودش را دارد. ماژول‌ها فقط از طریق `service` عمومی یکدیگر را صدا می‌زنند و import-linter این مرز را در CI کنترل می‌کند.

پشته‌ی فنی

بخش	انتخاب
زبان و ابزار	Python 3.13، uv
وب	FastAPI، Pydantic v2
داده	PostgreSQL 16 با PostGIS، SQLAlchemy 2 async، asyncpg، Alembic
امنیت	argon2 برای رمز، PyJWT، چرخش refresh token
فایل	S3 API، MinIO در توسعه و تست
کیفیت	ruff، mypy، import-linter
تست	pytest، httpx، testcontainers، Playwright
مانیتورینگ	Prometheus، Grafana، Loki، Tempo، OpenTelemetry، Grafana Alloy

چرا Redis نداریم

Redis در نسخه‌ی اول نیست، چون کاری نداریم که آن را بهتر از PostgreSQL انجام دهد و هر سرویس اضافه هزینه‌ی راه‌اندازی، پایش و تست دارد.

کاربرد رایج Redis	راه‌حل این نسخه
قفل رزرو	محدودیت `EXCLUDE` در PostgreSQL
نگه‌داشت موقت	ردیف `pending` با `hold_expires_at`؛ باید در دیتابیس باشد تا `EXCLUDE` آن را ببیند
Idempotency	`UNIQUE (idempotency_key)` در همان تراکنش رزرو
صف کارها	worker با دو کار ساده‌ی دقیقه‌ای
نشست و توکن	جدول `refresh_tokens`
Rate limit	شمارنده‌ی ساده در PostgreSQL برای ورود و ثبت‌نام

Redis وقتی ارزشمند می‌شود که rate limit سراسری با چند نمونه‌ی API و ترافیک بالا، کش جستجوی پرترافیک، یا صف واقعی برای ایمیل و پردازش عکس لازم شود. افزودن آن تغییر کوچکی است، چون پشت یک interface در `core` قرار می‌گیرد. منطق رزرو در هیچ حالتی به Redis وابسته نمی‌شود.

پایگاه داده

مبنا همان اسکیمای ۱۰ جدولی طراحی‌شده روی PostgreSQL 16 است و دو جدول و پنج اصلاح به آن اضافه می‌شود. از این پس migrationهای Alembic منبع حقیقت‌اند و DBML، ERD، `schema.sql` و README با آن‌ها همگام می‌مانند.

جدول‌ها

ماژول	جدول	نقش
identity	`users`	یک جدول برای مهمان و میزبان
identity	`refresh_tokens` (جدید)	هش توکن، انقضا و ابطال
identity	`auth_attempts` (جدید)	شمارنده‌ی rate limit ورود و ثبت‌نام
catalog	`cities`، `countries`	فهرست کنترل‌شده‌ی مقصدها
catalog	`amenities`، `amenity_categories`	کاتالوگ امکانات
listings	`listings`	آگهی، ظرفیت، قیمت و مختصات
listings	`listing_amenities`، `listing_photos`	عکس‌ها و امکانات هر آگهی
bookings	`bookings`	تاریخ، مهمانان، قیمت کپی‌شده و وضعیت

ماژول	جدول	نقش
reviews	`reviews`	امتیاز و متن نظر

اصلاح‌های migration اول

اصلاح	مشکلی که حل می‌کند
یکتایی روی `lower(email)`	`A@x.com` و `a@x.com` دو حساب جدا نمی‌شوند
`DEFERRABLE` روی `UNIQUE (listing_id, sort_order)`	جابه‌جایی ترتیب عکس‌ها و تغییر کاور خطا نمی‌دهد
ستون `cities.timezone`	«امروز» و زمان پایان اقامت به وقت همان شهر محاسبه می‌شود
ایندکس جزئی روی `hold_expires_at` برای `pending`	worker کل `bookings` را اسکن نمی‌کند
ستون public_location از نوع geography با ایندکس GiST	جستجوی شعاعی سریع و دقیق، بدون لو رفتن مختصات واقعی

قوانین موجود که حفظ می‌شوند

- `EXCLUDE USING gist` برای نبود هم‌پوشانی رزروهای `confirmed`، `pending` و `completed`
- `UNIQUE (idempotency_key)` برای امن بودن تکرار رزرو
- تریگر الزام `completed` برای نظر و تریگر همگام‌سازی امتیاز
- کلید خارجی ترکیبی `reviews → bookings`
- `CHECK (pets_allowed OR max_pets = 0)`

طراحی API

همه‌ی مسیرها زیر `/api/v1` هستند. FastAPI مستندات را خودکار می‌سازد: Swagger UI در `/docs`، ReDoc در `/redoc` و schema در `/api/v1/openapi.json`. در Swagger با دکمه‌ی Authorize و access token می‌توان همه‌ی endpointها را امتحان کرد. جزئیات فیلدها و نمونه‌ها همان‌جا آمده است.

قرارداد API: همین schema در فایل `backend/openapi.json` کنار کد نگه داشته می‌شود و فرانت کلاینتش را از آن می‌سازد. یک تست، کد و این فایل را مقایسه می‌کند و با هر اختلافی شکست می‌خورد. دلیلش این است که هیچ مسیر، فیلد یا خطایی بی‌خبر عوض نشود و فرانت بی‌صدا خراب نشود. بعد از تغییر عمدی، `caravansara export-openapi` فایل را دوباره می‌سازد و تغییر قرارداد در PR دیده و بررسی می‌شود. روش ساخت همچنان code-first است و فقط خروجی آن قفل شده، چون نوشتن spec جدا از کد برای این اندازه‌ی پروژه کار تکراری است.

ماژول	متد و مسیر	کاربرد	دسترسی
identity	`POST /auth/register`	ثبت‌نام	عمومی

ماژول	متد و مسیر	کاربرد	دسترسی
identity	`POST /auth/login`	ورود	عمومی
identity	`POST /auth/refresh`	چرخش توکن	refresh token
identity	`POST /auth/logout`	ابطال توکن	refresh token
identity	`GET`، `PATCH /me`	پروفایل	کاربر
catalog	`GET /countries`، `GET /cities?q=`	مقصدها و پیشنهاد خودکار	عمومی
catalog	`GET /amenities`	کاتالوگ دسته‌بندی‌شده	عمومی
search	`GET /search/listings`	شهر یا شعاع، کی، چند نفر و امکانات	عمومی
search	`GET /search/filters`	شمارش امکانات	عمومی
listings	`GET /listings/{id}`	جزئیات آگهی	عمومی
listings	`POST /listings`، `GET /me/listings`	فهرست و ساخت آگهی	کاربر
listings	`PATCH /listings/{id}`	ویرایش و غیرفعال کردن	میزبان
listings	`PUT /listings/{id}/amenities`	تنظیم امکانات	میزبان
listings	`POST /listings/{id}/photos/upload-url`	دریافت presigned URL	میزبان
listings	`POST /listings/{id}/photos`	ثبت عکس بعد از آپلود	میزبان
listings	`PATCH`، `DELETE /listings/{id}/photos/{photo_id}`	ویرایش و حذف عکس	میزبان
listings	`PUT /listings/{id}/photos/order`	تغییر ترتیب و کاور	میزبان
bookings	`POST /bookings`	ساخت نگه‌داشت با `Idempotency-Key`	کاربر
bookings	`GET /bookings/{id}`، `GET /me/bookings`	سفرها و جزئیات	مهمان یا میزبان
bookings	`POST /bookings/{id}/pay`	پرداخت شبیه‌سازی‌شده	مهمان
bookings	`POST /bookings/{id}/cancel`	لغو	مهمان یا میزبان
bookings	`GET /listings/{id}/bookings`	رزروهای یک آگهی	میزبان

ماژول	متد و مسیر	کاربرد	دسترسی
reviews	`POST /bookings/{id}/review`	ثبت نظر	مهمان
reviews	`GET /listings/{id}/reviews`	فهرست نظرها	عمومی

قرارداد خطا

همه‌ی خطاها با یک شکل برمی‌گردند، با کد ماشینی پایدار و متن انگلیسی تا کلاینت خودش پیام فارسی را نشان دهد.

```
{"error": {"code": "dates_unavailable", "message": "These dates were just booked."}}
```

وضعیت	کد	زمان برگشت
409	`dates_unavailable`	خطای `23P01` از `EXCLUDE`
409	`hold_expired`	پرداخت بعد از پایان مهلت
409	`invalid_booking_state`	گذار وضعیت غیرمجاز یا لغو از روز ورود به بعد
409	`already_reviewed`	نظر دوم برای یک رزرو
422	`idempotency_key_reused`	همان کلید با محتوای دیگر یا از کاربر دیگر
422	`capacity_exceeded`	بیشتر از ظرفیت مهمان یا حیوان خانگی
403	`own_listing`	رزرو آگهی خود میزبان
403	`review_window_closed`	نظر بعد از ۱۴ روز
429	`too_many_attempts`	عبور از rate limit ورود و ثبت‌نام
422	invalid_search_area	نبود شهر و شعاع، یا شعاع خارج از بازه‌ی مجاز

راهبرد تست

همه‌ی تست‌های دیتابیس روی PostgreSQL 16 واقعی با testcontainers اجرا می‌شوند، چون مهم‌ترین قوانین در خود دیتابیس هستند. معیار CI حداقل ۹۰٪ پوشش کد است.

لایه	چه چیزی را ثابت می‌کند
Unit	ماشین حالت رزرو، محاسبه‌ی قیمت، منطقه‌ی زمانی، جابه‌جایی پین
Integration	همه‌ی قوانین دیتابیس و هر endpoint، شامل خطاها، دسترسی و جستجوی شعاعی کنار خط ۱۸۰ درجه؛ پذیرش نظر در روز چهاردهم و رد آن در روز پانزدهم به وقت شهر؛ برقرار ماندن رزروها پس از غیرفعال شدن آگهی؛ آزاد دیده شدن اقامتگاه در جستجو پس از پایان مهلت نگه‌داشت؛ CORS فقط برای مبدأهای مجاز

لایه	چه چیزی را ثابت می‌کند
همزمانی	۲۰ درخواست هم‌زمان با نتیجه‌ی دقیقاً ۱ رزرو، تکرار هم‌زمان با یک کلید، مسابقه‌ی پرداخت با انقضا، لغو در برابر پرداخت؛ و کد بازنده‌ی هر رقابت: ثبت‌نام هم‌زمان با یک ایمیل، دو نظر برای یک اقامت، لغو در برابر تغییر هم‌زمان، و کلید تکراری که به خطای یکتایی می‌رسد
Jobs	انقضای نگه‌داشت، completed شدن به وقت شهر حتی در دو روز تغییر ساعت تابستانی، اجرای امن دو نمونه‌ی worker
آپلود	جریان کامل presigned URL روی MinIO واقعی، و رد شدن فایل بزرگ‌تر از ۱۰ مگابایت یا با نوع دیگر
Migration	اجرای `downgrade` و دوباره `upgrade` روی یک دیتابیس جدا، تا بازگرداندن در شرایط اضطراری امتحان‌شده باشد
قرارداد API	یکسان بودن `openapi.json` در ریپو با API ساخته‌شده از کد
معماری	مرز ماژول‌ها با import-linter
مانیتورینگ	`request_id` در هر پاسخ و لاگ، حذف ایمیل و توکن و فیلدهای محرمانه از لاگ، الگوی route به‌جای مسیر واقعی در متریک، شمارش رویدادهای رزرو و رزرو تکراری متوقف‌شده، trace با spanهای SQL، و ثبت نشدن trace برای `/health`
پیکربندی مانیتورینگ	داشبورد و هشدارها فقط متریک‌هایی را به کار می‌برند که کد واقعاً می‌سازد، و هر متریک برنامه روی داشبورد هست؛ هشدارها تست واحد `promtool` دارند و CI پیکربندی Prometheus و Alloy را بررسی می‌کند
داده‌ی دمو	بارگذاری روی دیتابیس خالی و توقف روی دیتابیس پر؛ ۹ شهر، ۷ کشور و ۶ ارز؛ ۳ تا ۸ عکس برای هر آگهی با ترتیب پیوسته و ابعادی برابر فایل در MinIO؛ هر ۷ حالت مرزی از راه API؛ متن فارسی؛ و ماندن داده وقتی بارگذاری عکس شکست بخورد
داده‌ی حجیم	تولید یکسان با seed و تاریخ یکسان، توزیع قیمت و نوع ملک نزدیک داده‌ی واقعی، رزروهای فعال بدون هم‌پوشانی، بارگذاری همه‌یاهیچ (قطع وسط کار ردیفی باقی نمی‌گذارد)، و قفل‌های ایمنی (production، دیتابیس تنظیمات، `--replace`)
E2E (Playwright)	هر ۳۰ endpoint روی سیستم کامل `docker compose`، از بیرون و با HTTP واقعی: image ساخته‌شده، تنظیمات واقعی، آپلود عکس به MinIO، ۶ رزرو هم‌زمان با دقیقاً یک برنده، و سناریوی کامل میزبان و مهمان؛ ۴۸ تست در `e2e/`

تست‌ها داده‌ی خود را با `Factory` در `backend/tests/factories.py` می‌سازند: کاربر و آگهی از راه API، و وضعیت‌هایی که API عمداً نمی‌سازد (اقامت گذشته، نگه‌داشت منقضی، اقامت پایان‌یافته) با درج مستقیم که همه‌ی قیدها و triggerها روی آن اجرا می‌شوند. هیچ تستی برای ساخت وضعیت رزرو SQL مستقیم نمی‌نویسد، جز تست‌هایی که خود قانون دیتابیس یا لحظه‌ی رقابت را می‌سازند. دلیل: تست نباید به داده‌ی دمو وابسته باشد، و SQL تکراری با هر تغییر جدول می‌شکند.

کدی که فقط بازنده‌ی یک رقابت اجرا می‌کند، با درخواست هم‌زمان عادی به‌ندرت اجرا می‌شود، چون بررسی اولیه معمولاً تکراری را زودتر پیدا می‌کند. برای همین هر تست دقیقاً لحظه‌ی رقابت را می‌سازد: بررسی اولیه چیزی نمی‌بیند و محدودیت

دیتابیس نوشتن دوم را متوقف می‌کند. هر جا معنی دارد، درخواست‌های هم‌زمان واقعی هم کنار آن اجرا می‌شوند.

تست‌های E2E کار تست‌های pytest را تکرار نمی‌کنند. pytest هر ماژول را با containerهایی که خودش می‌سازد تست می‌کند. E2E سیستم را همان‌طور که مستقر می‌شود می‌سنجد، تا خطای پیکربندی، Dockerfile یا اتصال به MinIO هم دیده شود. هر تست کاربر و آگهی خودش را می‌سازد تا تست‌ها موازی اجرا شوند و به هم وابسته نباشند، و داده‌ها پاک نمی‌شوند تا بعداً در دیتابیس دیده شوند. دو کاری که API در لحظه انجام نمی‌دهد، یعنی بردن یک اقامت به گذشته برای ثبت نظر و تمام کردن مهلت پرداخت، با SQL مستقیم انجام می‌شود.

داده‌ی آزمون و دمو

سه نوع داده برای سه کار ساخته می‌شود. این بخش نتیجه‌ی اولین کار با Spec Kit است (specs/001-test-mock-data/).

داده	برای	کجا	دلیل این شکل
دمو (`seed-demo`)	نمایش، آزمون دستی، UI	`db/seed_demo.sql` در git، روی دیتابیس خالی	کوچک و واقعی‌نما؛ ۷۹۰ آگهی در ۹ شهر، با همه‌ی حالت‌های مرزی
حجیم (`seed-bulk`)	اندازه‌گیری کارایی و ایندکس	دیتابیس جدای `caravansara_bulk`، بیرون از git	برنامه‌ریز PostgreSQL روی داده‌ی کم ایندکس را به کار نمی‌برد، پس اندازه‌گیری بدون آن گمراه‌کننده است
تست (`Factory`)	تست‌های بکند	ساخته در هر تست	تست نباید به داده‌ای وابسته باشد که با بازسازی دمو عوض می‌شود

- **دمو:**
- شش شهر Amsterdam، Lisbon، Copenhagen، Stockholm، London و New York City، با ۱۲۵ آگهی و تا ۱۵ نظر انگلیسی واقعی برای هر کدام. Inside Airbnb (CC BY 4.0):
- ۴۰ آگهی فارسی دست‌نوشته در تهران، اصفهان و شیراز، با قیمت ریالی، چون Inside Airbnb متن فارسی ندارد و رابط راست‌به‌چپ باید با متن واقعی فارسی (عدد و کلمه‌ی لاتین در جمله، نیم‌فاصله، متن خیلی بلند و خیلی کوتاه) آزموده شود.
- همه‌ی اشخاص ساختگی‌اند. نام میزبان واقعی در متن جایگزین می‌شود، و ساخت پیش از نوشتن فایل متوقف می‌شود اگر نام واقعی در نظری مانده باشد.
- **عکس‌ها:** ۸ تصویر ساختگی، یکی برای هر برچسب اتاق، که یک بار با Pillow ساخته و در git نگه داشته می‌شوند (حدود ۲۰۰ کیلوبایت). `seed-demo` آن‌ها را زیر کلید هر عکس در MinIO می‌گذارد. دلیل: عکس واقعی مال میزبان است، و تصویر ساختگی نه مشکل حق نشر دارد و نه به اینترنت نیاز دارد. MinIO در بارگذاری موازی گاهی درخواست را رد می‌کند (`ClientDisconnected`)، پس هر بارگذاری تا ۳ بار تکرار می‌شود. اگر storage در دسترس نباشد، داده می‌ماند و `seed-demo --photos-only` عکس‌ها را بعداً می‌گذارد.
- **حالت‌های مرزی:** نگه‌داشتی که ۴ دقیقه بعد از بارگذاری منقضی می‌شود، رزرو منقضی، اقامت ۳۶۵ شبه، ورود در آخرین روز مجاز، تحویل همان روز، آگهی غیرفعال با رزرو، و آگهی بدون نظر. شناسه‌ی هر کدام در `db/demo/README.md` آمده است. تاریخ‌ها نسبت به روز بارگذاری و به وقت شهر آگهی‌اند (`now() AT TIME`)

`ZONE tz)::date`)، چون `CURRENT_DATE` تاریخ UTC سرور است و نزدیک نیمه‌شب برای بعضی شهرها یک روز اختلاف دارد.

- **catalog افزایشی:** `caravansara seed` فقط ردیف‌های تازه را اضافه می‌کند (`ON CONFLICT DO NOTHING`)، تا شهرهای تازه بدون پاک شدن داده به دیتابیس موجود برسند. قبلاً با وجود هر ردیف رد می‌شد.
- **داده‌ی حجیم:**
- از `db/bulk/city_profiles.json` ساخته می‌شود: خلاصه‌ی آماری داده‌ی واقعی شش شهر (سهم نوع ملک، صدک‌های قیمت، ظرفیت و اندازه، احتمال هر امکان، امتیاز، شب‌های رزروشده در سال با `estimated_occupancy_l365d`، و نقطه‌های نمونه). پس ساخت آن به دانلود نیاز ندارد.
- هر آگهی RNG خودش را دارد، تا هر جدول جدا و جریانی تولید شود و با seed و `--today` یکسان، خروجی یکسان باشد.
- بارگذاری با `COPY` در یک تراکنش است، و همه‌ی قیدها و triggerها روشن می‌مانند، تا دیتابیس تولیدکننده را هم بسنجد.
- هرگز در دیتابیس تنظیمات برنامه بارگذاری نمی‌شود، و `--replace` فقط دیتابیسی را پاک می‌کند که نامش با `_bulk` تمام شود.
- اندازه‌گیری روی لپ‌تاپ توسعه (2026-11-05): ۱۰ هزار آگهی در ۱۹ ثانیه؛ ۱۰۰ هزار آگهی، ۳٫۰۳ میلیون رزرو و ۱٫۱۶ میلیون نظر در ۳ دقیقه و ۲۷ ثانیه، با حجم ۱٫۹ گیگابایت.
- **سقف ریال:** ستون‌های قیمت `NUMERIC(10,2)` حداکثر حدود ۱۰۰ میلیون را می‌پذیرند، و رزرو چندشبه با قیمت واقعی ریالی از آن رد می‌شود. پس داده‌ی دمو قیمت ریالی را پایین و رزروها را کوتاه نگه داشته است. پهن کردن ستون‌ها کار جدایی در `docs/future-work.md` است.

زیرساخت و تحویل

کل سیستم با یک فرمان `docker compose` بالا می‌آید: PostgreSQL 16، worker، API با PostGIS و MinIO. MinIO از سال ۲۰۲۵ دیگر تصویر Docker منتشر نمی‌کند، پس compose و تست‌ها از آخرین نسخه‌ی ثابت Bitnami (`bitnamilegacy/minio:2025.4.22`) استفاده می‌کنند. چون کد فقط با S3 API کار می‌کند، رفتن به سرور سازگار دیگری فقط تغییر تنظیمات است.

- **مخزن:** کد در `Caravansara` قرار می‌گیرد که از قبل یک مخزن git محلی است و پوشه‌ی `db/` را هم در خود دارد.
- **CI:** GitHub Actions روی هر push و pull request، ruff، mypy، import-linter، تست‌ها، ساخت تصویر Docker و بررسی پیکربندی مانیتورینگ را اجرا می‌کند.
- **تنظیمات:** از متغیرهای محیطی؛ کلیدها داخل مخزن ذخیره نمی‌شوند.
- **اسناد طراحی:** ERD، DBML، `schema.sql` و README با اصلاحیه‌ها همگام می‌شوند.
- **عکس‌ها:** کد با S3 API نوشته می‌شود؛ رفتن از MinIO به S3 یا R2 فقط تغییر تنظیمات است.

مانیتورینگ

هدف دیدن وضعیت سیستم در هر لحظه و پیدا کردن علت خطا بدون حدس است. سه نوع داده جمع می‌شود، چون هر کدام به سؤال متفاوتی جواب می‌دهد، و هر سه به هم وصل‌اند:

نوع داده	به چه سؤالی جواب می‌دهد	ابزار
متریک	چه مقدار، چه‌قدر سریع، چند خطا؟	Prometheus
لاگ	دقیقاً چه اتفاقی افتاد؟	Loki
trace	زمان یک درخواست در کدام بخش صرف شد؟	OpenTelemetry و Tempo
داشبورد و هشدار	همه‌ی موارد بالا در یک جا	Grafana و قانون‌های هشدار Prometheus

این ابزارها متن‌باز و رایگان‌اند و با Docker اجرا می‌شوند. برنامه فقط OpenTelemetry و قالب Prometheus را می‌شناسد، پس عوض کردن ابزار فقط پیکربندی پوشه‌ی `observability/` را تغییر می‌دهد، نه کد را.

در برنامه:

- **لاگ:** هر خط یک شیء JSON است با route ، `trace_id` ، `request_id` ، وضعیت و زمان پاسخ. ایمیل، token و فیلدهای محرمانه پیش از نوشتن پوشانده می‌شوند، چون لاگ معمولاً دسترسی گسترده‌تری از دیتابیس دارد و نشانی دقیق طبق همین سند خصوصی است. query string نوشته نمی‌شود.
- **شناسه‌ی درخواست:** هر پاسخ `X-Request-ID` دارد. شناسه‌ی ورودی از proxy فقط با شکل امن پذیرفته می‌شود تا نتوان با آن متن جعلی در لاگ نوشت.
- **متریک:** تعداد، خطا و زمان پاسخ برای هر endpoint با برچسب الگوی مسیر، مثل `/listings/{listing_id}` ، نه نشانی واقعی، چون هر مقدار یکتا یک سری زمانی جدا در Prometheus می‌سازد. رویدادهای کسب‌وکار هم شمرده می‌شوند: شروع نگه‌داشت، پرداخت، انقضا، لغو، پایان اقامت، رزرو تکراری متوقف‌شده با `23P01` ، برخورد با rate limit و نظر.
- **worker:** چون API ندارد، متریک‌هایش را روی پورت ۹۱۰۰ می‌دهد: تعداد اجرا، زمان آخرین اجرای موفق و مدت اجرا. healthcheck کانتینر هم همین پورت را بررسی می‌کند.
- **trace:** برای هر درخواست و هر اجرای worker، با span برای هر دستور SQL و هر فراخوانی S3. span های SQL بدون درخواست والد، مثل `SELECT 1` در health check، دور ریخته می‌شوند تا هر چند ثانیه یک trace بی‌معنی ساخته نشود.

در زیرساخت: همه با profile جدای `observability` در `docker compose` اجرا می‌شوند، تا محیط توسعه‌ی روزانه سبک بماند. Grafana Alloy تنها collector است: لاگ همه‌ی کانتینرها را از Docker می‌خواند و trace برنامه را با OTLP می‌گیرد. در Loki فقط `service` و `level` برچسب‌اند و `request_id` و `trace_id` به‌شکل structured metadata ذخیره می‌شوند، چون مقدار یکتا برای هر درخواست ایندکس را منفجر می‌کند. datasourceها و داشبورد از فایل‌های repo ساخته می‌شوند، تا هر محیط همان داشبورد را داشته باشد و تغییرش در git بررسی شود. از هر خط لاگ می‌شود trace آن را باز کرد و از هر span لاگ‌های همان درخواست را.

هشدارها:

هشدار	شرط	چرا مهم است
`ApiDown`	هیچ نمونه‌ی API تا ۱ دقیقه پاسخ ندهد	سایت از دسترس خارج است

هشدار	شرط	چرا مهم است
`WorkerDown`	worker تا ۲ دقیقه نباشد، حتی اگر کانتینرش حذف شده باشد	نگه‌داشت‌های پرداخت‌نشده قفل می‌مانند و هیچ خطایی در API دیده نمی‌شود
`WorkerStale`	آخرین اجرای موفق worker بیش از ۵ دقیقه پیش باشد	worker اجرا می‌شود ولی شکست می‌خورد
`HighErrorRate`	بیش از ۵٪ درخواست‌ها تا ۵ دقیقه 5xx بگیرند	
`SlowSearch`	p95 جستجو تا ۱۰ دقیقه بیش از ۱ ثانیه باشد	جستجو پرترافیک‌ترین صفحه است
`DatabaseDown`	PostgreSQL تا ۱ دقیقه در دسترس نباشد	
`DatabaseConnectionsHigh`	بیش از ۸۰٪ اتصال‌ها تا ۵ دقیقه در استفاده باشند	درخواست‌های تازه منتظر می‌مانند

آستانه‌ها حدس اول‌اند و بعد از چند هفته داده‌ی واقعی تنظیم می‌شوند، چون آستانه‌ی حدسی یا مدام هشدار می‌دهد یا هیچ‌وقت. ارسال هشدار به ایمیل یا پیام‌رسان به Alertmanager یا contact point در Grafana نیاز دارد و در این نسخه نیست.

در محیط واقعی: `/metrics` روی همان پورت API است و نباید از اینترنت در دسترس باشد (شبکه‌ی داخلی یا قانون proxy، یا `CARAVANSARA_METRICS_ENABLED=false`). `CARAVANSARA_OTEL_SAMPLE_RATIO` با بالا رفتن ترافیک فقط بخشی از traceها را نگه می‌دارد.

روش کار با GitHub Spec Kit

از این پس هر کار با GitHub Spec Kit انجام می‌شود: اول «چه و چرا» با معیار پذیرش، بعد طرح فنی، بعد فهرست کارها، و در آخر کد و تستی که همان معیارها را ثابت می‌کند. دلیل: تصمیم‌ها پیش از کد بررسی می‌شوند، کاری که با کمک هوش مصنوعی انجام می‌شود با یک سند نوشته‌شده سنجیده می‌شود، و سند و کد از هم فاصله نمی‌گیرند.

مرحله	فرمان در Claude Code	خروجی
اصول پروژه (یک بار، و هنگام اصلاح)	`/speckit-constitution`	`.specify/memory/constitution.md`
مشخصات: چه و چرا، با معیار پذیرش Given/When/Then	`/speckit-specify`	`specs/NNN-name/spec.md`
رفع ابهام، وقتی spec سؤال باز دارد	`/speckit-clarify`	پاسخ‌ها در همان `spec.md`
طرح فنی و بررسی با اصول پروژه	`/speckit-plan`	`plan.md` و سندهای طراحی همان کار

مرحله	فرمان در Claude Code	خروجی
فهرست کارها به ترتیب وابستگی	`/speckit-tasks`	`tasks.md`
بررسی هماهنگی spec، طرح و کارها پیش از کد	`/speckit-analyze`	گزارش، بدون تغییر فایل
پیاده‌سازی	`/speckit-implement`	کد و تست
مقایسه‌ی کد با سندها تا «Converged»	`/speckit-converge`	کارهای مانده در `tasks.md`

- **اصول پروژه:** constitution از تصمیم‌های همین سند نوشته شده است: قانون‌های حیاتی در دیتابیس، مونولیت ماژولار با مرز اجباری، قرارداد API پیش از کد، تست روی زیرساخت واقعی، حریم خصوصی، قابلیت پایش و سادگی. هر طرح پیش از کار با آن سنجیده می‌شود و هر استثنا باید دلیل داشته باشد. نسخه‌ی 1.1.0 (2026-11-05) دو استثنای پذیرفته‌شده در کار اول را به قاعده تبدیل کرد، تا کارهای بعدی دوباره به آن‌ها نیاز نداشته باشند. اول، بسته‌ی ابزار (مثل `devdata`) لایه‌های ماژول را لازم ندارد، ولی باید قرارداد import-linter داشته باشد و در production اجرا نشود. دوم، معیاری که زمان روی سخت‌افزار مشخص است با دست و طبق quickstart اندازه گرفته می‌شود، ولی بقیه‌ی همان معیار باید تست خودکار داشته باشد.
- **نسخه:** Spec Kit نسخه‌ی 1.2.3 است. آخرین نسخه‌ای انتخاب شد که دست‌کم دو هفته از انتشارش گذشته بود، چون این ابزار چند بار در هفته نسخه‌ی تازه می‌دهد. ارتقا با `specify integration upgrade claude` انجام می‌شود.
- **فایل‌ها:** `.specify/` قالب‌ها، اسکریپت‌ها و اصول پروژه را دارد و `.claude/skills/` فرمان‌های Claude Code را. spec هر کار در `specs/NNN-name/` و در همان pull request کد قرار می‌گیرد.
- **زبان:** spec، طرح، فهرست کارها و چک‌لیست‌ها به فارسی نوشته می‌شوند. عنوان بخش‌های قالب و شناسه‌ها (`T001`، `SC-001`، `FR-001`) همان‌طور که در قالب هست می‌مانند، چون `/speckit-analyze` و `/speckit-converge` نیازمندی‌ها و کارها را با همین‌ها پیدا می‌کنند. کد، مسیر فایل، فیلد API و کد خطا انگلیسی می‌مانند. خود constitution انگلیسی است، چون قالب‌ها و فرمان‌هایی که آن را می‌خوانند انگلیسی‌اند.
- **مرز با این سند:** این سند همچنان تنها منبع حقیقت معماری است. تصمیم‌های هر کار بعد از پایان آن به این سند منتقل می‌شوند، چون دو منبع حقیقت دیر یا زود با هم تناقض پیدا می‌کنند.
- **استثنا:** تغییری که رفتار سیستم را عوض نمی‌کند، مثل اصلاح متن یا ارتقای وابستگی، بدون Spec Kit انجام می‌شود.
- **pull request:** قالب `.github/pull_request_template.md` چک‌لیست spec، تست معیارهای پذیرش و به‌روزرسانی سندها را دارد.

مراحل پیاده‌سازی

هر مرحله با تست‌های خودش تمام می‌شود؛ رزرو زودتر از جستجو می‌آید چون جستجو به آن وابسته است. همه‌ی مراحل انجام شده‌اند. با داده‌ی آزمون و دمو، اکنون ۲۰۲ تست روی PostgreSQL و MinIO واقعی با پوشش ۹۸٫۰٪ وجود دارد.

1. اسکلت پروژه، migration، CI، Docker، git اول و seed
2. `identity` و `catalog`، همراه با rate limit ورود و ثبت‌نام

3. `listings` و عکس‌ها
4. `bookings` و تست‌های هم‌زمانی
5. `jobs`
6. `reviews`
7. `search`
8. همگام‌سازی اسناد و داده‌ی دمو

خارج از دامنه و ریسک‌ها

این موارد عمداً در نسخه‌ی اول نیستند و بعضی از آن‌ها پیش از انتشار واقعی لازم می‌شوند.

مورد	اثر نبودنش
پرداخت واقعی و بازپرداخت	برای انتشار واقعی ضروری است؛ فعلاً شبیه‌سازی می‌شود
تأیید ایمیل و بازیابی رمز	کاربری که رمز را فراموش کند راهی برای بازگشت ندارد؛ نیاز به سرویس ایمیل دارد
تقویم مسدودسازی تاریخ توسط میزبان	میزبان فقط می‌تواند کل آگهی را غیرفعال کند
قیمت متفاوت برای هر شب	قیمت شبانه ثابت است
ویرایش تاریخ رزرو	مهمان باید لغو کند و دوباره رزرو کند
حذف حساب کاربری	به سیاست داده و حریم خصوصی نیاز دارد
تبدیل ارز	هر آگهی با ارز خودش نمایش داده می‌شود
پیام مهمان و میزبان، علاقه‌مندی‌ها، پاسخ به نظر	طبق تصمیم دامنه‌ی ساده حذف شده‌اند
پنل ادمین	کاتالوگ‌ها فقط با seed تغییر می‌کنند

ریسک‌های شناخته‌شده:

- **جستجوی شعاعی:** شعاع بزرگ همراه فیلتر تاریخ ممکن است کند شود؛ سقف ۱۰۰ کیلومتر این را محدود می‌کند، ولی باید با بار واقعی اندازه‌گیری شود.
- **ابعاد عکس:** عرض و ارتفاع را کلاینت اعلام می‌کند و بکند فقط وجود فایل را بررسی می‌کند.
- **فایل‌های یتیم:** آپلودی که ثبت نشود در فضای ذخیره‌سازی می‌ماند؛ با قانون lifecycle پاک می‌شود.